

Container Images Bit By Bit

Approximate schedule

1. 10:00 - Intro & Some History (20 minutes)
2. 10:20 - Lab 0 (optional) & Lab 1 (30 minutes)
3. 10:50 - Philosophy Corner: What *are* containers? (10 minutes)
4. 11:00 - Lab 2 + break (30 minutes)
5. 11:30 - Image building with dedicated tools (10 minutes)
6. 11:40 - Lab 3 (20 minutes)
7. 12:00 - Base image strategies discussed; final thoughts (15 minutes)
8. 12:15 - Lab 4 (15 minutes)
9. 12:30 - Lab 5 (optional) and Q&A (30 minutes)

Slides

A PDF of the presentation slides will be posted in the `slides` directory after the workshop. The PDF will include a link to the live slideshow on the web.

Lab setup

- The labs assume a recent version of a Red-Hat-family (RHEL, Fedora, Rocky, etc.) Linux distribution. If you are not using such a distribution, you may need to adjust some commands, e.g. to use `apt` instead of `dnf` to do package installs, but you should still be able to complete all the labs with most other major distributions (Debian, Ubuntu, etc.)
 - The lab instructions for running commands assume Rocky Linux 10.
 - Your lab system will need to have `podman` and `ltd` installed. (Most systems will already have `ltd` but you may need to install `podman` yourself.)

- The `terraform` directory contains a Terraform configuration suitable for using Terraform or OpenTofu to spin up an AWS instance pre-configured to work through the lab exercises. You do not need to use this to create a lab system suitable for running the labs but it may make it easier to get started. The default login user for the Rocky AMI is `rocky` and by default the config creates a new AWS keypair and writes the private key file in the `files` subdirectory in the Terraform config directory (the path to the private key will be one of the outputs).

Lab exercises

Lab 0: Image Prehistory

The original form of "containing" processes was simply confining them to only seeing a subset of the entire filesystem -- "changing the root" or `chroot`. Chrooting a process at the command line is as simple as using the "chroot" utility.

Let's set up a directory with a minimal base system and a shell. (If you're using a lab system set up with the supplied Terraform code, this block has already been done.)

```
# Assume root identity
sudo su -
# Write some info to a file in the normal filesystem
date > /usr/local/share/hello-scale
# Create the directory under the chroot that we will write some data to
mkdir -p /usr/local/chroot/usr/local/share
# Add some basic utilities to this directory
dnf install --installroot /usr/local/chroot --releasever 10 \
    util-linux-core iproute procps-ng coreutils
# Write some info to a file under the chroot directory
uname -a > /usr/local/chroot/usr/local/share/hello-scale
```

Now let's run a chrooted shell.

```
# Assume root identity if you aren't already
sudo su -
# Let's look at a file on the real root filesystem
cat /usr/local/share/hello-scale
# As root, run a new shell in a chroot
chroot /usr/local/chroot bash
# Now let's look at our hello-scale file again... but something is different
cat /usr/local/share/hello-scale
# Note, however, that nothing else about this process is really confined.
# We can still see network info for the host system
```

```
ip addr
# To see process info we mount the /proc filesystem
mount -t proc proc /proc
ps aux
# Exit the chrooted shell
exit
# We can now see the second hello-scale file in its real filesystem path
cat /usr/local/chroot/usr/local/share/hello-scale
```

Lab 1: Container images == tarballs

If a process can be confined to a subset of the underlying filesystem, this suggests that that subset could be rolled up into a tidy archive and distributed as a unit, and in fact this is one way you can build the image for a container. Of course to do this, you want the size of the resulting archive to be as small as possible. One way to do this is to include only the specific binaries you need and only the specific libraries they depend on. Let's try installing the `bash` binary in a chroot, but this time we'll just copy the binary and work through the library dependencies to make it work.

```
# First, create a new directory in /usr/local
mkdir -p /usr/local/chroot-minimal/usr/bin
# Now copy the bash binary to it
cp $(which bash) /usr/local/chroot-minimal/usr/bin
# Now try running it as a chroot
chroot /usr/local/chroot-minimal bash
```

It doesn't work -- what's going on? Although the error is a little obscure, what's wrong is that the `bash` binary is linked to shared libraries that we haven't copied yet, so they can't be loaded by a process chrooted to this directory. Fortunately the tools exist to show us which libraries we need:

```
ldd $(which bash)
```

The `ldd` command will tell you the libraries `bash` is linked to, and their paths. Copy each one to the corresponding path in the `chroot-minimal` directory:

```
mkdir -p /usr/local/chroot-minimal/lib64
cp /lib64/libtinfo.so.6 /usr/local/chroot-minimal/lib64/libtinfo.so.6
```

```
cp /lib64/libc.so.6 /usr/local/chroot-minimal/lib64/libc.so.6
cp /lib64/ld-linux-x86-64.so.2 /usr/local/chroot-minimal/lib64/ld-linux-x86-64.so.2
```

Now try chrooting again:

```
chroot /usr/local/chroot-minimal bash
```

This time you should get a shell prompt. (Note, however, that `bash` is *all* you have this time. Try running `ps`, for example.)

Exit the chroot:

```
exit
```

If you have time left over before the workshop resumes, try doing the same with the `ip` binary so you can run `ip addr` inside this chroot.

Lab 2: Container images as minimal distros

Notice that in Lab 0, the `--installroot` flag was used to install `bash` in the chrooted directory. This is a much less tedious way to reach the end goal we worked toward in lab 1: Rather than tracing layers of shared library dependencies with `ldd`, and being dependent on what is already installed in the filesystem, you can use a package manager (or other similar utilities like `buildroot`) to do the heavy lifting. Your image may not be quite as small, but maintaining and updating it will be much easier -- this is a key tradeoff in managing container images.

Let's do a tarball-style image again, but this time we'll use a package manager to install `nginx` in our target directory, then we'll import it as a container image.

```
mkdir -p /usr/local/chroot-pkg/
dnf install -y --installroot /usr/local/chroot-pkg --releasever 10 nginx
cd /usr/local/chroot-pkg
tar zcvf /root/nginx-chroot.tar.gz .
podman import /root/nginx-chroot.tar.gz
```

This by itself successfully creates an image, but is lacking in several respects when it comes to actually running the container:

- There is no default entrypoint, so if we try to run it, the container runtime will throw an error unless we manually specify an entrypoint (the default command to run when the container starts).
- There are no tags, so the only way to run the container is by knowing its hash.
- There is no metadata to say what ports should be exposed or volume mount points used by default.

If you want to see real examples of how the official `nginx` container is built, check the project's GitHub repo.

Lab 3: Using OCI image building tools

The main thing that separates a modern container image from a simple tarball is the metadata embedded in it, including things like the default process (if any), the ports intended to be exposed, the volume mounts, and any tags applied to the image. It's possible, but tedious, to hand-build an image to contain these things -- it's much simpler to use tools built for the purpose. In this lab, we'll use `podman` to build our `nginx` image again. This time we'll use a build file ("Dockerfile", or more generically, Containerfile) to instruct `podman` how to build the image.

First, create a build file environment on your system:

```
mkdir /root/nginx-custom
```

Now, in `nginx-custom`, create a file called `containerfile` with these contents:

```
FROM rockylinux/rockylinux:10

RUN dnf clean all
RUN dnf makecache
RUN dnf update -y
RUN dnf install -y nginx
```

Now, let's build our image:

```
podman build --tag nginx-scale23x:v1 .
```

If you have some extra time in this lab, try doing the same with other services.

Lab 4: Monoliths vs. layers

One of the advantages of using tools for image building instead of the DIY approach, is that they automatically enable features in the OCI spec intended to make images easier to work with. In terms of building and using images, layers are one of the most important features keeping storage usage from ballooning.

Let's add a new package install to our custom `nginx` image. Edit your `containerfile` and add this at the bottom:

```
RUN dnf install -y jq
```

Now build the image with a `v2` tag:

```
podman build --tag nginx-scale23x:v2 .
```

And finally, examine the layers that make up both images:

```
podman image inspect nginx-scale23x:v1 | jq .[].RootFS.Layers
podman image inspect nginx-scale23x:v2 | jq .[].RootFS.Layers
```

The fact that five of the six layers in the second image are shared with the first image allows the build to be faster and the storage consumed to be smaller.

Layering also allows faster image pulls. Pull the official `nginx:1.29.5` image:

```
podman pull nginx:stable
```

Notice the layers being pulled down to make up the image. Now pull the `latest` image tag and watch again as the layers are processed:

```
podman pull nginx:latest
```

Note that some layers are skipped: they already were downloaded for the `1.95.2` image, so they are reused in place.

Lab 5: Inspecting the image in action

This lab is more open-ended -- the goal is to give you a chance (and some pointers) to look under the hood of where and how the container image is turned into a running system.

Run the official `nginx` container:

```
sudo su -  
podman pull nginx:1.29.5  
podman run nginx:1.29.5
```

Now explore the area where `podman` extracts the image layers before it runs the container.

- Run `mount | grep "overlay on"` . What do you see in the output?
- What do you see when browsing the directories associated with the running `nginx` container?
- Compare the `merged` directory with the directories that make it up.

Extra credit

If you're feeling curious, run the lab Terraform config again (or for the first time, if you used your own lab system) with the argument `-var create_freebsd_instance=true` and then log in to the new instance as `ec2-user` using the SSH private key file listed in the output. Once you're logged in, read the documentation on FreeBSD jails and try creating one to run `nginx` .

Links to tools:

- Podman and Buildah
- Packer
- Buildroot